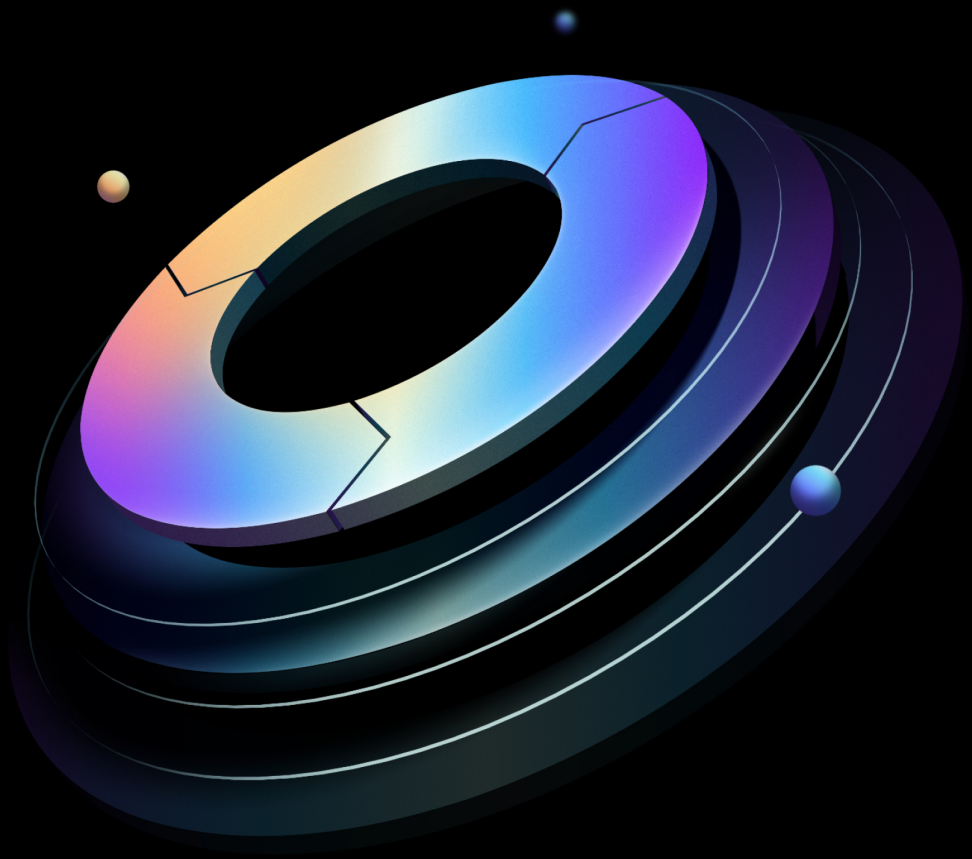
HashiCorp

Nomad User Guide

Built from commit 3a6a26c83c

HashiCorp | Validated Designs

09 November 2026



HashiCorp

Contents

1 Introduction **4**
1.1 Why use HashiCorp Validated Designs? . . . 4
1.2 Use cases covered . . . 4

2 Self-service application deployment **5**
2.1 Advanced deployment strategies . . . 5
2.2 Rolling upgrades . . . 6
2.3 Single canary deployment . . . 8
2.4 Blue/green deployments . . . 10
2.4.1 Recommendations . . . 11
2.5 Workload dependencies . . . 12
2.5.1 Lifecycle block . . . 12
2.5.2 Template block . . . 13
2.6 Health Checking . . . 13
2.7 External dependencies . . . 14

3 Containerized orchestration **15**
3.1 Docker . . . 15
3.2 Podman . . . 15

4 Non-containerized orchestration **15**
4.1 Exec2 . . . 15
4.2 Raw exec . . . 16

5 Job placement **16**
5.1 Node pool selection . . . 17
5.2 Task resource sizing . . . 17

6 Networking **17**
6.1 Considerations . . . 18

7 Application scaling **18**
7.1 Resource allocation with resource blocks . . . 19
7.2 Horizontal application scaling with Nomad Autoscaler . . . 20

7.3 Horizontal scaling strategies . . . 20
7.3.1 Threshold . . . 20
7.3.2 Target value . . . 21
7.3.3 Pass-through . . . 21
7.3.4 Fixed value . . . 22
7.4 Scaling policy examples . . . 23
7.4.1 Spread considerations . . . 26
7.5 Dynamic application sizing . . . 27
7.6 Stateful workload considerations . . . 28
7.6.1 Storage . . . 28
7.6.2 Networking . . . 28
7.6.3 State consistency . . . 28
7.6.4 Clustered applications . . . 28
7.6.5 Kill timeout . . . 29
7.6.6 Service discovery . . . 29
7.7 Standardization . . . 29

8 Service discovery 30
8.1 Health checks . . . 31
8.1.1 HTTP health checks . . . 31
8.1.2 TCP health check . . . 33
8.2 Upgrades . . . 35
8.3 Leveraging service discovery in tasks . . . 35

9 Secrets and variables 38
9.1 Understanding and implementing Nomad Enterprise variables and secrets . . . 38
9.2 Nomad versus HCL variables . . . 39
9.3 HCL variables . . . 40
9.4 Choosing between Nomad variables and HCL variables . . . 41
9.5 Nomad variables versus Vault variables . . . 42
9.6 How to use variables . . . 43
9.6.1 Creating a variable . . . 43
9.6.2 ACLs . . . 44
9.6.3 Accessing variables from tasks . . . 45
9.6.4 Locks . . . 46

10 Changelog **47**
10.1 2026-08 47
10.2 2026-01 47
10.3 2025-10 47
10.4 2025-07 47
10.5 2025-05 47
10.6 2025-04 47
10.7 2025-02 47

11 Credits **48**

1 Introduction

> **Note**
>
> These guides were reorganized in August 2026. The previous Solution Design Guide and Operating Guides (Adoption, Standardization, Scaling) have been replaced with three role-based guide types: Installation, Administration, and User. Read the HVD 2.0 release notes for full details on what changed and where to find the content you need.

1.1 Why use HashiCorp Validated Designs?

HashiCorp Validated Designs (HVD) provide practitioners with opinionated guidance for achieving production-grade deployments of HashiCorp products. These designs are purpose-built for delivering foundational use cases, with a baseline level of architectural and operational maturity. They draw on the field experiences of Solutions Engineers and Solutions Architects working with customers across a wide range of environments and organizational requirements.

Each guide provides access to an opinionated reference architecture, including key design decisions and the rationale behind them. Where applicable, guides identify modular design components that you can adjust to align with organizational or regulatory requirements without compromising the overall integrity of the implementation. For many deployments we include Terraform modules to automate large portions of infrastructure provisioning and software installation.

1.2 Use cases covered

> **Note**
>
> The Installation Guide and Administration Guide cover responsibilities such as installation, access-control design, upgrades, backups, observability setup, policy enforcement, quotas, scheduler configuration, and control plane scaling.

This guide covers the following consumption workflows:

- Deploying applications through self-service workflows.
- Running containerized and non-containerized workloads.
- Influencing job placement and task scheduling from job specifications.
- Configuring workload networking and service ports.

- Scaling applications with Nomad Autoscaler patterns.
- Registering and consuming service discovery data.
- Using Nomad variables and secrets from workloads.

2 Self-service application deployment

Nomad Enterprise offers a powerful and flexible platform for self-service application deployment, catering to a wide range of workloads and deployment strategies.

As an individual deploying applications into Nomad, focus primarily on your application's requirements instead of the infrastructure hosting your application. If any of the topics discussed below are not met, establish a clear communication channel with the Nomad platform operators to address these gaps.

Implement a standardized workflow to facilitate a smooth self-service experience This approach eliminates guesswork regarding available resources, networking options, and storage solutions.

It is *imperative* that you and your team feel equipped to provide a powerful self-service solution for deploying your applications to Nomad in time for the commissioning of production deployments. Use with the topics discussed under Initial Configuration and those below as inputs.

2.1 Advanced deployment strategies

Nomad Enterprise offers modern deployment strategies for your long running services that allow for seamless updates, risk mitigation, and minimal downtime. These strategies support both container and non-container workloads.

While rolling upgrades offer an excellent starting point for organizations embarking on their modernization journey, the ultimate goal for many is to implement sophisticated Blue/Green deployment models.

Nomad Enterprise's flexible approach allows teams to evolve their deployment practices gradually, adapting to increasing complexity and operational maturity over time.

2.2 Rolling upgrades

Rolling upgrades gradually replace instances of the old version with the new version and if there is an issue during deploying, use auto rollback to minimize service disruption.

Use this strategy for the early stages of Nomad Enterprise adoption as it balances efficiency and risk, making it suitable for many applications, but it can be slower for large deployments and requires careful management of multiple versions during the transition.

Rollbacks are straightforward to execute and crucial for minimizing downtime, however, they do not prevent the initial impact of a deployment. Do not rely upon this for 0% disruption.

To elaborate, if a user has a sticky session with an instance and a rolling update then replaces it, the session may not redirect to the newer instance. Instead, it is terminated.

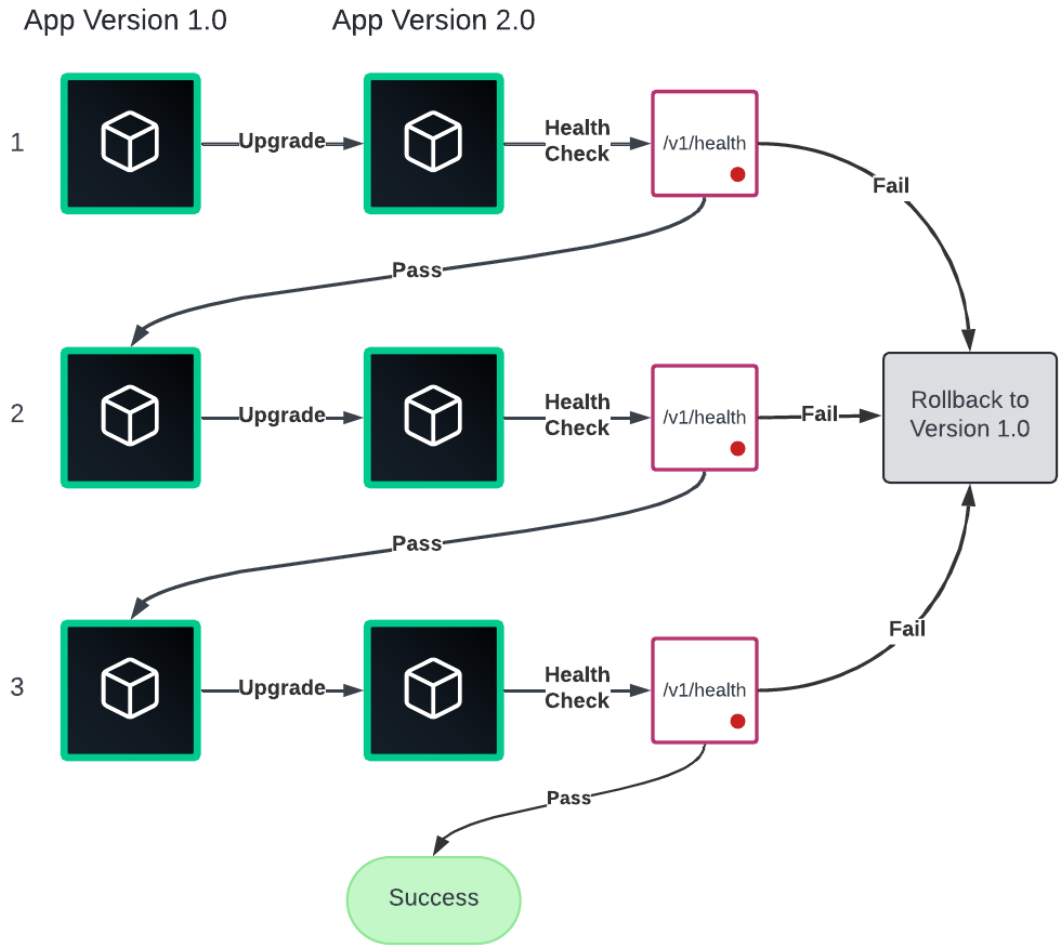


Figure 1: IMAGE

View Rolling updates tutorial for more information on enabling this feature in your jobs.

Recommendations

- View `update` documentation for all available options
- Set `auto_revert` to **`true`** to revert and rollback to the latest stable job version.

> **Note**
>
> Consider if this could cause issues for your stateful workloads such as databases. If so, set this to `false`.

- Set your health checks and timeouts correctly for your tasks since the `update` block uses the `checks` block within a task to ensure a task is healthy before its promoted. See service discovery section for more details on configuring the `service.check` block
- Use `stagger` and `max_parallel` to ensure a minimal disruption.
- You can view the status of the rolling update by view the landing page of your job.

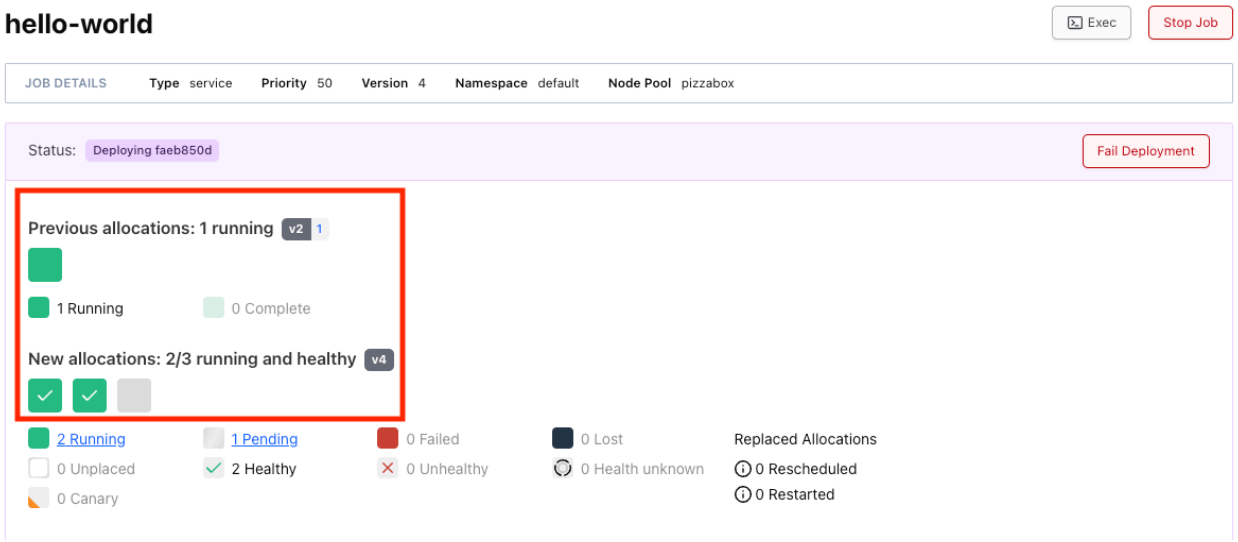


Figure 2: IMAGE

2.3 Single canary deployment

Canary deployments involve releasing a new version instance to a small subset of users, without affecting the existing workloads. Accomplish this by using a load balancer or proxy that can dynamically update instances and set various endpoint weights.

Consul Enterprise is a great option by using a service router with service mesh. When you confirm there are no errors, you promote the older instances, by performing a rolling upgrade.

Canary deployments are a great next step after rolling upgrades. Single canary deployments come with the same risks as rolling upgrades along with the small sample size of canary deployments can sometimes lead to misleading performance or usage metrics.

Use this single instance as a production grade test before the final rolling upgrade.

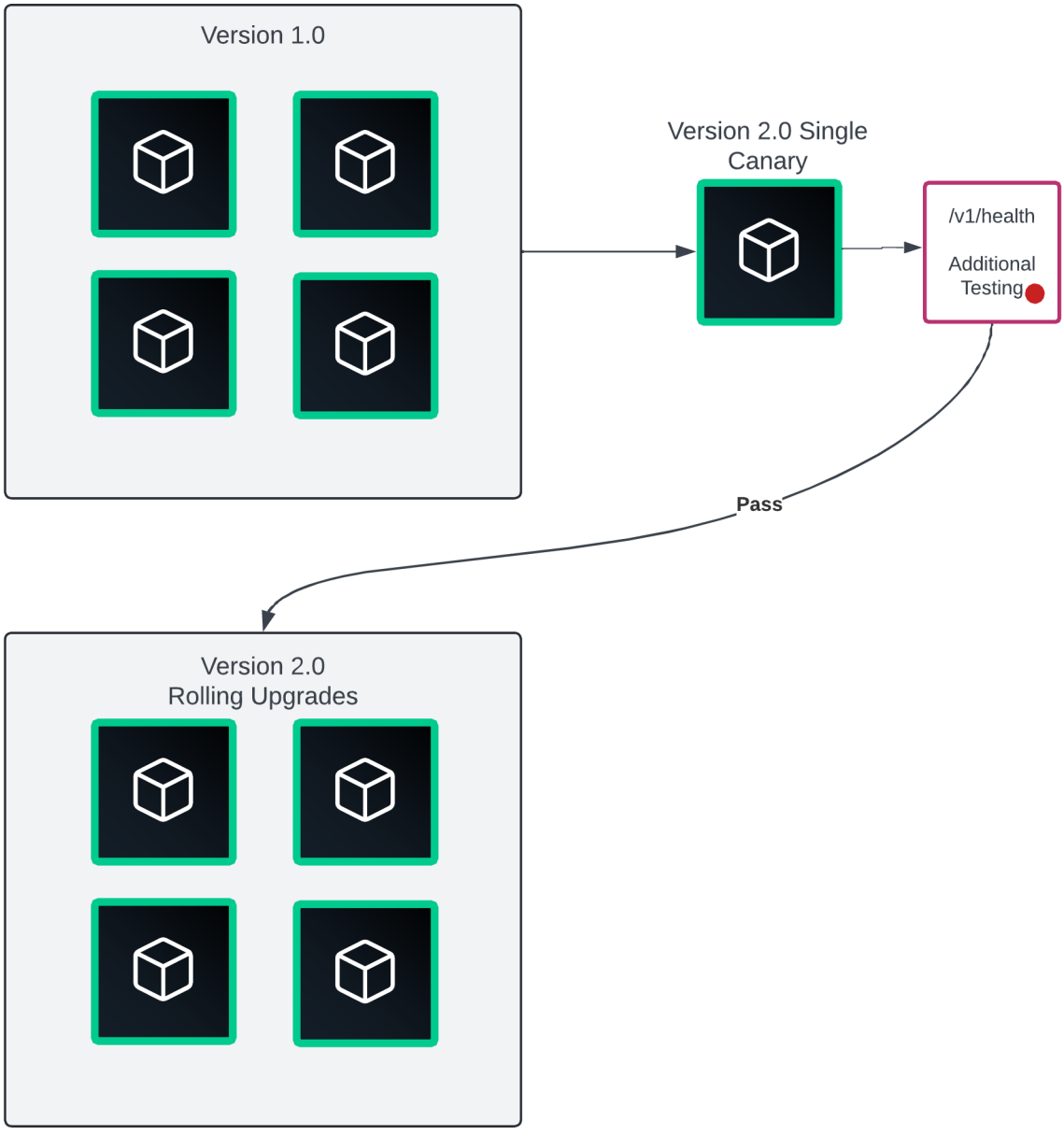


Figure 3: IMAGE

View the Deploy with Canaries tutorial for more information on how to enable this capability for your jobs.

If using Consul Enterprise service mesh, visit Deploy Seamless Canary Deployments with Service Splitters tutorial to learn how to leverage Consul for B/G and Canary deployments.

Recommendations

- Properly evaluate the subset of users who are the best candidates for testing and are swift to report any issues.
- Use Consul or another third party load balancer or proxy to steer traffic during the upgrade.

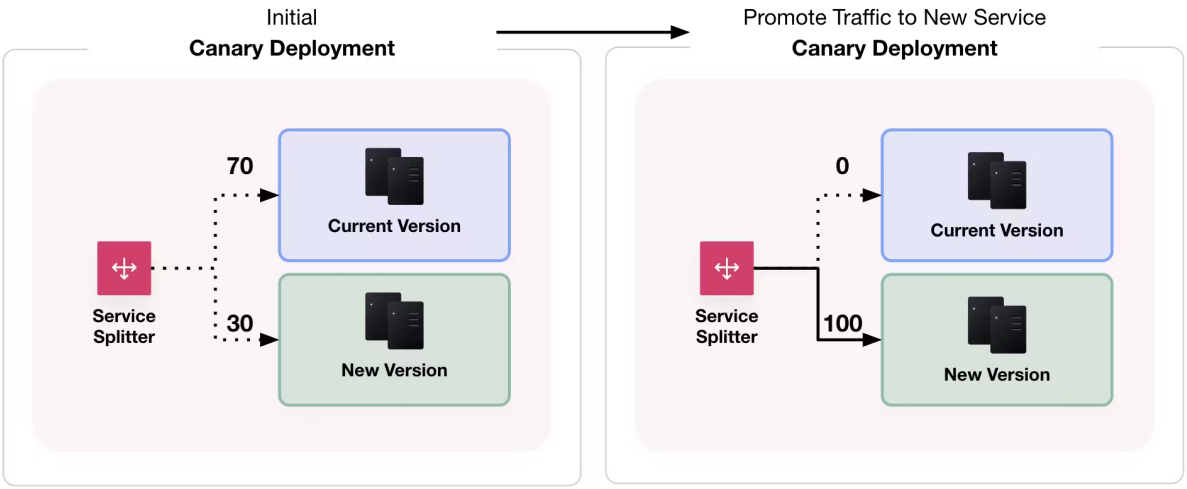


Figure 4: IMAGE

- Use Manual Promotion if required for tier 1 critical workloads.
- Use automated rollback mechanisms to fail the canary deployment if you receive certain metrics.

2.4 Blue/green deployments

After implementing single canary deployments, Blue/green deployments are the last and best step in your deployment strategy. In a blue/green deployment, there are two application versions.

Only one application version is active at a time, except during the transition phase from one version to the next. The term "active" tends to mean *receiving traffic* or *in service*. B/G deployments scale the best and provide zero downtime deployments.

Similar to single canary, accomplish this by using a load balancer or proxy that can dynamically update instances and set various endpoint weights. Consul is a great option by using a service router with service mesh.

When you confirm there are no errors, you cut over traffic incrementally to the newer instances.

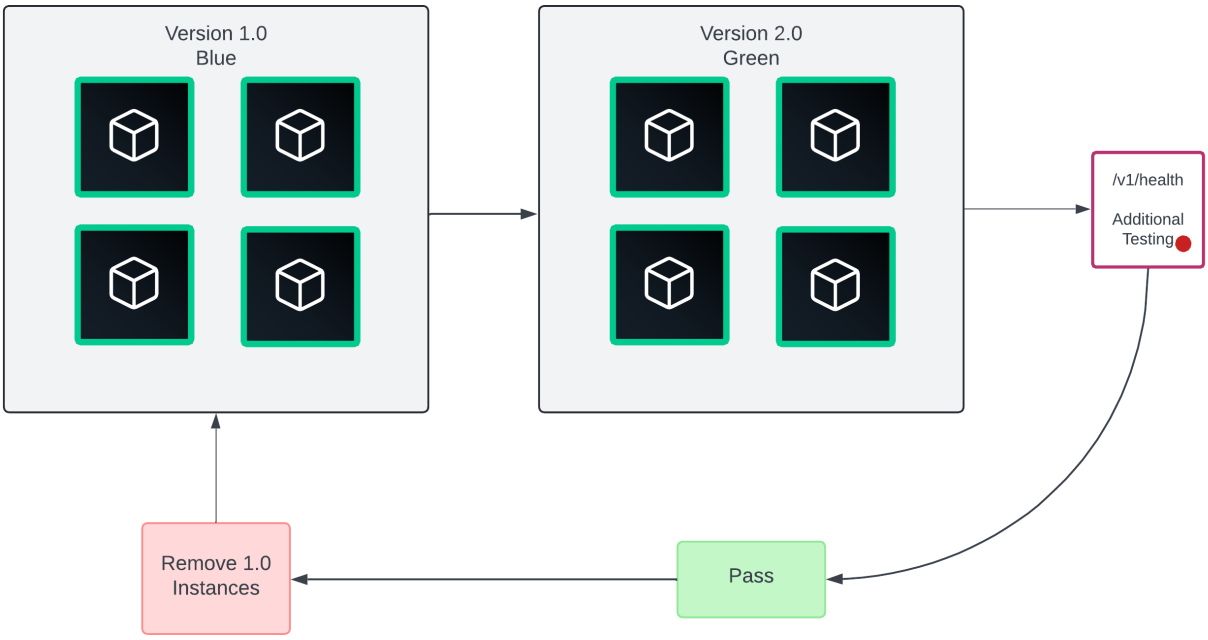


Figure 5: IMAGE

Visit the Blue/Green Deployments tutorial to learn how to implement.

If using Consul Enterprise service mesh, visit Deploy seamless canary deployments with service splitters tutorial to learn how to leverage Consul for B/G and Canary deployments.

2.4.1 Recommendations

- If feasible, consider implementing blue/green deployments at the start of your project.

> Tip
>
> This approach depends on having a load balancer or proxy that can split and route Layer 7 traffic, which might not be available in your environment.If these prerequisites are not met, begin with the rolling upgrades strategy as you develop your canary and blue/green upgrade strategies.

- Instead of an entire cut-over, consider gradually shifting traffic from blue to green. This strategy helps mitigate risks by limiting the potential impact on the broader user base, allowing for more controlled observation and easier rollback if issues arise.
- Implement automated testing of the new instances before switching users over to the new instances. If your automated testing is sufficient, consider using the auto_promote flag.
- Leverage monitoring and alerting to reveal any issues when new users move over to the new instances.

2.5 Workload dependencies

Task dependencies occur when one task relies on another task, job, or external resource to be running or available. There are several patterns to manage these dependencies to ensure your application is dynamic and resilient.

2.5.1 Lifecycle block

Used to express task dependencies in Nomad Enterprise by configuring when a task runs within the lifecycle of a task group. Main tasks are tasks that do not have a `lifecycle` block.

- Init Tasks - A task that completes and exits before proceeding with the main task
- Sidecar Pattern - A task that starts after the main tasks, but stays alive and runs along side the main task
- Cleanup Task - A task that runs after the main tasks have stopped. They are useful for performing post-processing that is not available in the main tasks.
- Leader Task - use to specify if the task is the leader of the group. If used, all other tasks are dependent on the leader. Meaning, the leader task stops first, followed by non-sidecar and non-poststop tasks, and then sidecar tasks. Once this process completes, Nomad Enterprise triggers post-stop tasks.

2.5.2 Template block

As it relates to task dependencies, `templates` are useful for generating dynamic configuration data from other workloads such as:

- IP address and/or dynamic ports of a service
- Retrieve KV data from Consul or Vault
- A tasks metadata values
- Retrieve job or client node related variables and use them as environment variables such as the job name, datacenter name, or the bridged network IP address

Use templates to reference another task within the job or you expect data of the upstream job to change. View service discovery section for more information on templating.

Warning

Dependency is the term for each item in templates that read from Vault Enterprise, Consul Enterprise, or Nomad Enterprise. All the `template` blocks share the same internal runner which de-duplicates dependencies requesting the same item. Avoid having large numbers of dependencies for a given task, as each dependency requires at least one concurrent request (a possibly blocking query) to the upstream server.If a task has more than 128 dependencies, a warn-level log appears in the Nomad Enterprise client logs which reports 'watching this many dependencies could DDoS your servers', referring to the Vault Enterprise, Consul Enterprise, or Nomad Enterprise cluster queried.

Tip

For templates that read from Vault Enterprise, Consul Enterprise, or Nomad Enterprise, there are settings called respectively vault_retry, consul_retry and nomad_retry. Configure how each Nomad client retries these services in cases of unavailability.The default behavior, which includes killing allocations after a number of failed retries, might not suit every environment.

2.6 Health Checking

The service discovery section details health checking for your services to ensure they are running properly.

However, there are some task dependency specific recommendations:

- Implement a reasonable retry attempt within your application before sending exit signals.
- The built in health checking covers most use cases, however if it is not sufficient, consider implementing circuit breakers within your application code or through Consul service mesh
- You can run multiple health checks for a service. One could check itself, and another could run a script that runs a custom binary command you may use in your environment.

```
service {
  check {
    name     = "HTTP Check"
    type     = "http"
    port     = "api"
    path     = "/v1/health"
    interval = "5s"
    timeout  = "2s"
  }
  check {
    type    = "script"
    command = "/bin/bash"
    args    = ["-c", "/bin/some-binary", "some args"]
  }
}
```

See service discovery health check section for more detailed explanations and recommendations.

2.7 External dependencies

When dealing with external dependencies outside of Nomad, you may need to consider several strategies to ensure their applications are resilient and can handle potential issues with these dependencies. Here are some recommendations:

- `artifact` block The artifact block allows you to download external resources before starting a task. This is useful for fetching configuration files, libraries, or other dependencies. Do not use along side templates, as the artifacts are only retrieved at runtime.

```
artifact {
    source = "https://example.com/app-config.json"
    destination = "local/config.json"
}
```

- Avoid using hard-coded and static data.

Tip

Leverage service discovery to reference details about that service via templates.

3 Containerized orchestration

3.1 Docker

Nomad supports Docker workloads natively as it comes bundled with Nomad. Refer to these resources to configure the Docker task driver and use it in your jobs:

- Configure the Docker task driver
- Use the Docker task driver in a job

3.2 Podman

Unlike Docker, Nomad operators need to download the Podman driver separately and ensure its availability on the client's nodes.

Podman supports OCI containers and its command-line tool aims to be a drop-in replacement for Docker and support most of the same options as Docker.

To use Podman in a job task, refer to the task configuration section of the Podman task driver documentation.

4 Non-containerized orchestration

In addition to containers, Nomad supports non-containerized workloads as well. Below we discuss the various task drivers that can be used and recommendations for each.

4.1 Exec2

Used to execute a command for a task. It offers a security model optimized for running 'ordinary' processes with low startup times and minimal overhead in terms of CPU, disk, and memory utilization. It also uses kernel features such as the Landlock LSM, cgroups v2, and theunshare system

utility. This is great for running batch jobs, legacy workloads, or other workloads that are executable on the underlying host.

- The exec2 task driver is not built into Nomad Enterprise. Install this using a package manager or downloaded onto the client host in the configured plugin directory.
- Refer to the plugin page for client requirements and capabilities
- By default the exec2 driver exposes a set of default paths. Customize these or specify additional, allowable paths at the plugin level, which applies to all tasks making use of the exec2 task driver, or at the task level, which applies specifically to each task.
- If a path is not exposed within the driver, it cannot feature in job files.

4.2 Raw exec

Unlike Exec2, use the raw exec driver to execute a command for a task without any isolation. Further, the task started is as the same user as the Nomad process. As such, use it with extreme care and it is off by default. Consider raw exec under any of the following conditions.

- Are confident in your node isolation and security posture
- Do not like the overhead of managing paths with Exec2
- Prefer to manage cgroups

For both exec2 and raw exec, Nomad Enterprise is effectively executing a command as if you were on the terminal for that machine. Any dependencies must exist on the host. Consider the tradeoff with having standardized client images and node pools, versus using lifecycle blocks and artifact blocks within your job files. Meaning, if a job has a large list of dependencies, it may take a long time to download and clean up all the artifacts needed. Refer to the [dependency] section for more information.

5 Job placement

Nomad scheduler defaults, node pools, preemption, and memory oversubscription are configured and managed by platform operators — see the Administration Guide: Scheduling administration.

Workload authors can influence placement by using job-level configuration that the platform team exposes for application teams.

5.1 Node pool selection

Jobs are able to opt in to a node pool by specifying the `node_pool` parameter. Node pool configurations override the default scheduler configuration. For example, if setting the default cluster configuration to `binpack` and there is a node pool with `spread` set, then any workloads placed on the node pool use `spread`.

5.2 Task resource sizing

Tasks must specify `memory_max` to take advantage of memory oversubscription.

```
job "example-job" {
  group "example" {
    task "server" {
      resources {
        cpu        = 100
        memory     = 256
        memory_max = 768
      }
    }
  }
}
```

To avoid degrading the cluster experience, coordinate with platform operators on resource reservations, memory limits, and resource quotas before deploying production workloads.

6 Networking

> **Note**
>
> Nomad networking modes and cluster-level network prerequisites are configured and managed by platform operators — see the Administration Guide: Networking configuration.

As Nomad Enterprise is a workload orchestrator, it focuses on the scheduling aspects of a deployment, touching areas such as networking little. Make a choice of networking mode based on infrastructure requirements and organizational policies. Regardless of the chosen networking mode, it is crucial to ensure that all necessary networking configurations are present and available **before**

deploying jobs. If there is a mistake in the networking configuration, when attempting to deploy jobs, errors are likely.

6.1 Considerations

- For self-service models, it is ideal for application owners to have templated job definitions by Nomad Enterprise operators with the correct networking mode and port mappings.
- Coordinate with your network engineers regarding any necessary security groups or firewall configurations to ensure proper network access for your workloads.
- Consider your port mapping strategy. Use dynamic port allocation to avoid conflicts in shared environments.
- Familiarize yourself with the network plugins available in your Nomad Enterprise cluster. Different plugins may offer features like network policy enforcement or advanced routing capabilities.
- If using Docker or Podman drivers, leverage their native networking capabilities, including custom overlay networks for multi-host communication.
- The Container Network Interface (CNI) plugin allows for more advanced networking configurations and is useful in multi-node clusters. It enables the use of various network plugins for different networking requirements.
- For more information on networking in Nomad, refer to the Nomad networking documentation.

7 Application scaling

Application scaling is a critical aspect of operating Nomad Enterprise as a shared service. It enables organizations to efficiently manage resources, maintain application performance, and optimize costs in dynamic environments. As workloads fluctuate, the ability to automatically adjust resources becomes paramount for ensuring operational efficiency.

Nomad Enterprise offers multiple scaling mechanisms to address the diverse needs of modern applications. These include horizontal scaling, which adjusts the number of task instances, and dynamic application sizing, which modifies resource allocations for individual tasks.

7.1 Resource allocation with resource blocks

Resource blocks in Nomad job specifications are crucial for ensuring optimal resource allocation and utilization. The definition of resource requirements for tasks guarantees the allocations are on nodes that have sufficient capacity to run your applications.

Focus on these primary and associated attributes when configuring required resources for your workload.

- `cpu` & `memory` - resources{}
- `disk` - emphemeral_disk{}
- `device type` - device{}
- Always specify CPU and memory requirements for each task. Allocate slightly more resources than the default minimum (CPU=100Mhz & Memory=300mb) to account for peak loads and prevent resource contention.
    - Nomad automatically finds a suitable node with the available resources you specify. Ensured this before considering horizontal scaling.
- Conduct thorough performance testing to determine precise resource requirements for your applications. Dynamic application sizing along with correct stress testing can give you an idea of required resources for your application.
- emphemeral_disk{} is only used if you have data within the allocation itself. This does not include any mounted volumes or bind paths outside of the allocation directory. Ephemeral disks are perfect for data that you can rebuild if needed, such as an in-progress cache or a local copy of data.
    - If your application data does not need to be ephemeral, however you need larger allocation disk size, consider using volumes.
    - If your workload has specific device requirements, consider using the device{} block to request access to a GPU, FPGA, or TPU.
- Use Numa settings for your application if needed.

7.2 Horizontal application scaling with Nomad Autoscaler

Horizontal scaling involves adding or removing instances of an application to handle varying loads. Manually increase the `count` parameter using UI, command-line tool, or API. This determines the number of instances within a `group`. Manual adjustments may be sufficient during your early Nomad adoption journey, but as your footprint grows and processes refine, we recommend you use Nomad's Autoscaler. The Autoscaler can dynamically adjust the number of instances based on predefined policies.

At a high level, the Autoscaler operates with three key components.

1. APM (Application Performance Monitoring): Collects metrics from either Nomad API, Prometheus, or Datadog. Nomad Enterprise operators configured this on the client node configuration file and in the Autoscaler job specification. Prometheus is the recommended APM. See the Observability section.
2. Strategy: Determines how to interpret metrics and make scaling decisions.
3. Target: Executes the scaling actions on the Nomad cluster. For application horizontal scaling, this block can be omitted as Nomad will populate them on job submission.

For more details, go to Nomad Autoscaler concepts page.

7.3 Horizontal scaling strategies

The Nomad Autoscaler provides several strategies for horizontal application scaling, each designed to address different scaling scenarios and requirements. This section details four strategies along with their use cases and recommendations to assist you in choosing the right strategy.

7.3.1 Threshold

The threshold strategy defines a range of metric values with a lower and upper bound, and takes action when the current metric value falls within those bounds. Note that `lower_bound` is inclusive while `upper_bound` is exclusive. For example, a check with `lower_bound = 70` and `upper_bound = 100` with `delta = 1` adds instances when the metric is between 70 and 99.

Use cases

- Maintaining metrics within an acceptable range (for example memory usage 40-80%).
- Applications where sudden spikes in demand require handling.
- Useful for managing resources efficiently by scaling down during low demand periods.

Recommendations

- Set appropriate upper and lower bounds based on application behavior and requirements.
- Use wider thresholds for more stable scaling, narrower for more responsive scaling.
- Combine with appropriate cool-down periods to prevent rapid scaling oscillations.
- Tune thresholds based on historical and forecasted data (increased demand) to ensure optimal scaling.

7.3.2 Target value

The Target Value strategy aims to maintain a specific metric at a desired target value. Unlike the Threshold strategy, which is reactionary, the target value strategy is proactive. For example, keeping CPU utilization at 70%. Nomad adjusts the number of instances to achieve this target.

Use cases

- Applications with dynamic workloads where maintaining a specific performance metric is crucial.
- Helps in optimizing costs by scaling resources based on actual demand, avoiding over-provisioning.

Recommendations

- Select the metric that best represents the application's performance and resource needs.
- Use with metrics that have a clear correlation to instance count (for example CPU usage).
- Implement appropriate cool-down periods to prevent oscillation.

7.3.3 Pass-through

The Pass-through strategy allows external systems or custom logic to dictate the number of instances. Nomad Enterprise acts as an executor, scaling the application based on the input it re-

ceives, removing computational power from the Nomad Enterprise cluster nodes.

Use cases

- Use custom scaling logic such as integrating with external monitoring tools.
- Direct mapping of metrics to instance count (for example one instance per active user).
- Scaling based on external systems or custom metrics.

Recommendations

- Ensure the metric directly correlates to the desired instance count.
- Implement safeguards (max_scale_up/down) to prevent rapid over or under-scaling.
- Ensure robust integration and failure testing between Nomad and the external system to avoid discrepancies in scaling decisions.

7.3.4 Fixed value

Used for client node scaling and not relevant to application scaling.

When choosing strategy, understand your workload thoroughly and select a strategy that aligns with your application's behavior and scaling needs. Start with conservative scaling strategies and as you fine tune and observe the scaling behavior, consider combining multiple strategies for more nuanced scaling decisions. Regular monitoring and adjustment of your scaling configurations are essential as your application and workload evolve.

Be mindful of the resource implications to ensure your cluster can handle the potential maximum scale and implement safeguards by setting appropriate minimum and maximum values to prevent under or over-scaling, and use suitable cool-down periods to avoid rapid scaling oscillations. Implement [sentinel], [quotas], and [node pool governance] as guardrails.

Thorough testing, including simulations of various load scenarios, is vital to verify the effectiveness of your scaling strategies.

Evaluation interval and cool-down period

Evaluation interval and cool-down period are parameters that impact the responsiveness and stability of your scaling operations.

Evaluation Interval

How often Nomad Autoscaler looks at your metrics to decide if it needs to scale up or down. It affects how your system can respond to changes, how often it might scale, and how much work the Autoscaler itself has to do. Set it too short, and you might be scaling unnecessarily often. Too long, and you might miss important spikes and cause performance degradation.

When choosing an interval, consider the following.

- In most cases, the default settings will be sufficient, however critical and volatile workloads may require shorter intervals for quicker responses.
- Shorter intervals (for example 5 to 30 seconds) increase the load on the Autoscaler and metric sources, however it can lead to more accurate scaling decisions, optimizing resource utilization for the applications.
- Use longer intervals (for example 5 to 10 minutes) if there are resource constraints on the Autoscaler or if your application can tolerate slower response times to workload changes. This can help reduce the load on the Autoscaler while still meeting the scaling needs of less time-sensitive applications.

Cool-down period

The cool-down period is a wait time enforced after each scaling action, during which Nomad Enterprise prevents new scaling operations. Serving to prevent rapid successive scaling actions, this allows time for the system to stabilize after a scaling event, and reduce unnecessary scaling actions due to temporary spikes or dips in metrics.

When choosing a cool-down period, consider the following.

- Should be longer than the time it takes for new instances to become fully operational.
- For workloads with predictable patterns, a longer cool-down period (for example 10 to 15 minutes) can be effective. For more volatile workloads, a shorter period (for example 2 to 5 minutes) might be necessary.
- Consider how long it takes for metrics to stabilize and reflect the impact of a scaling action.

7.4 Scaling policy examples

Below are basic and common scaling policies which we aim to be sufficient as a starting point as you refine your scaling logic over time.

Memory and CPU scaling

```
job "app" {
  datacenters = ["dc1"]

  group "app" {
    count = 1

    scaling {
      enabled = true
      min     = 1
      max     = 10

      policy {
        cooldown = "10m"
        evaluation_interval = "5m"

        check "memory_usage" {
            source = "prometheus"
            query = "nomad_client_allocs_memory_allocated{alloc_id=\"${NOMAD_ALLOC_ID
                }\",task_group=\"${NOMAD_GROUP_NAME}\"}"

        group = "resource_usage"
            strategy "threshold" {
                upper_bound = 512
                lower_bound = 400
          delta       = 1
            }
        }
        check "cpu_usage" {
            source = "prometheus"
            query = "nomad_client_allocs_cpu_allocated{alloc_id=\"${NOMAD_ALLOC_ID
                }\",task_group=\"${NOMAD_GROUP_NAME}\"}"

        group = "resource_usage"
            strategy "threshold" {
                upper_bound = 1000
                lower_bound = 700
          delta       = 1
            }
        }
      }
    }
    # ...
  }
}
```

In the preceding definition, Nomad Enterprise adds instances when memory usage is within 400-512 MB (lower_bound inclusive, upper_bound exclusive) or when CPU usage is within 700-1000 MHz, until reaching the defined `max` count. When using multiple checks with the threshold strategy, the `group` parameter ensures all checks work together properly for scale-down operations. Scaling uses the safest check action if there is overlap between the bounds.

Nomad Enterprise sets the `evaluation_interval` to a conservative 5 minutes to respond to changes in traffic and sets the `cooldown` period to 10 minutes to allow the system to stabilize after each scaling action.

Custom metrics

```
job "app" {
  datacenters = ["dc1"]

  group "app" {
    count = 1

    scaling {
      enabled = true
      min     = 3
      max     = 10

      policy {
        cool-down = "90s"
        evaluation_interval = "15s"

        check "request_rate" {
            source = "prometheus"
            query = "sum(rate(http_requests_total{}[1m]))"

            strategy "target-value" {
                target = 100
            }
        }
      }
    }
    # ...
  }
}
```

This example shows an API service which requires an aggressive scaling policy using a custom `prometheus` metric that is specific to that application. A scaling event (either up or down) triggers

if required, to keep the target value of 100 requests per minute with a minimum of 3 instances. A short evaluation interval (15 seconds) allows reaction to traffic changes and a minimum of 3 instances ensures high availability. Also, set the `evaluation_interval` to 15 seconds to respond quickly to changes in traffic. If we assume a well-tested and stable API service, we can confidently set the `cooldown` to 90 seconds to stabilize after each scaling action over a short period of time.

7.4.1 Spread considerations

The `spread` block allows application owners to increase the failure tolerance of their applications. Specify a node attribute to spread allocations over. This allows operators to spread allocations over attributes such as datacenter, availability zone, or even rack in a physical datacenter using metadata.

In a self-service environment, end users deploying applications into Nomad may not always be aware of cluster-level spread configurations. As a best practice, it is advisable to include a `spread` block in your job file when you have specific distribution requirements. This approach ensures you meet your application's distribution needs, regardless of any cluster-level settings.

Note

When scaling down (reducing the number of allocations), the Autoscaler does not consider the spread configuration or metadata when choosing which allocations to remove. Instead, it uses allocation IDs to determine the order of removal.For example, if you have:
Allocations 1, 2, 3 on Node 1
Allocations 4, 5, 6 on Node 2
Allocations 7, 8, 9 on Node 3
When scaling down, Nomad Enterprise removes allocations in reverse order (9, 8, 7, and so on) rather than maintaining an even distribution across nodes. is behavior can potentially impact the availability and balance of your application across the cluster during scale-down operations.

- When you have specific distribution requirements, always include a `spread` block in your job file. This ensures you meet your requirements regardless of cluster-level configurations.
- Be aware of the current limitation in scale-down operations. You may need to implement additional logic or monitoring to maintain desired distribution during scale-down events.

- You can use multiple spread blocks to create more complex distribution strategies. For example, spreading across both datacenters and rack IDs.
- Monitor the actual distribution of your allocations to ensure they align with your intended spread configuration.

7.5 Dynamic application sizing

Dynamic application sizing (DAS) allows Nomad to provide CPU and memory recommendations to tasks based on their actual usage. This feature complements horizontal scaling by optimizing resource utilization at the individual task level.

The `scaling` stanza is the same for horizontal scaling. It applies, however, to the task level for vertical scaling using DAS and three additional strategies.

Refer to the Dynamic App Sizing Concept and Dynamic App Sizing tutorial for an overview of the concepts, how to configure, and recommendations.

While DAS does not automatically apply recommended resource changes to tasks, you can implement automation to streamline the process and reduce manual intervention. Consider the following approaches if you are confident in your metrics and DAS configuration.

- If the native Nomad Enterprise UI is not sufficient, then you can utilize the Nomad API to retrieve recommendations and display them in a custom dashboard for efficient monitoring and decision-making.
- Implement alerts based on DAS recommendations or significant resource changes. Direct these alerts to Nomad Enterprise operators, application owners, or both, ensuring swift awareness of potential optimizations.
- Incorporate DAS recommendations into your deployment process for new versions to ensure they receive the latest recommended resource allocations.
- Configure DAS individually for each group or task, rather than through a global setting that affects all jobs within your Nomad environment, so that there is no single, overarching setting that can enable DAS across all jobs simultaneously. This means you must modify each job specification in your deployment pipeline to incorporate the appropriate DAS configuration blocks.

7.6 Stateful workload considerations

When implementing autoscaling for stateful applications in Nomad Enterprise, consider the following factors to ensure data integrity and performance. Be aware that certain stateful applications may have inherent limitations on how they scale. Be aware that some stateful applications may not be a good fit for autoscaling.

7.6.1 Storage

The underlying storage is *crucial* for stateful applications when autoscaling. For container workloads, leverage Nomad Enterprise's Container Storage Interface (CSI) support to dynamically attach storage to instances as your workload scales. This gives you the flexibility to manage storage resources on-the-fly, adapting to your application's changing needs.

7.6.2 Networking

While networking is generally less of a concern when autoscaling within the same Nomad Enterprise cluster, some stateful applications require stable network identities (for example IP addresses) for client connections or inter-node communication. Be aware of these requirements when designing your scaling strategy and collaborate with your networking team.

7.6.3 State consistency

Synchronize new instances with the current state of the application, which can be time-consuming and resource-intensive. Consider the performance impact on your stateful application during scale events and plan accordingly.

Also implement conservative scaling policies that allow sufficient time for data synchronization and state management between scaling events.

7.6.4 Clustered applications

For clustered applications, utilize variable locks to prevent split-brain scenarios and data corruption during scale-up and scale-down events.

7.6.5 Kill timeout

Ensure your application has proper shutdown procedures to persist state in a safe manner before scaling down. Use `kill_timeout` to allow sufficient time for graceful shutdowns and consider `max_kill_timeout` set on the client configuration to ensure job authors do not exceed this amount.

7.6.6 Service discovery

Implement Service Discovery or Service Mesh to dynamically manage network identities and route traffic appropriately.

As always, use a well-tested autoscaling development environment to identify and resolve any issues before implementing in production. This is *critical* for stateful applications due to their data persistence requirements. For additional details not related to autoscaling, view the Considerations for Stateful Workloads page.

7.7 Standardization

To promote a self service model, consider implementing job templates or utilizing Nomad Pack to provide a consistent deployment workflow to include standard metrics across applications while still allowing for customization.

An alternative method involves creating a global scaling policy within the Autoscaler agent configuration. However, it is important to recognize that a one-size-fits-all approach may not be suitable for all applications.

In many cases, it is more effective to allow job authors to determine the specific scaling requirements for their applications.

Operators can then use features such as Sentinel, resource quotas, and node pool governance to establish appropriate guardrails. This balanced approach ensures flexibility for individual application needs while maintaining overall system integrity and resource management.

8 Service discovery

Service discovery is the process of automatically detecting and locating services within your infrastructure. In the context of Nomad Enterprise, it allows applications and services to find and communicate with each other without hard-coding IP addresses and ports.

Service discovery is a more advanced form of DNS that goes beyond translating domain names into IP addresses. It provides additional capabilities such as real-time updates, health status information, metadata about services, load balancing, and support for dynamic environments.

- Real-time updates as services come online or go offline
- Health status information about services
- Metadata about services (for example version, capabilities)
- Load balancing across multiple instances of a service
- Support for dynamic environments where service locations frequently change

These advanced features make service discovery an essential component in modern distributed systems, enabling flexible and resilient application architectures. In a shared service environment, where multiple teams deploy various applications and microservices, service discovery becomes crucial for several reasons.

- Services may scale up or down, or move between hosts frequently. Service discovery ensures that clients can always locate the services they need, regardless of changes in the underlying infrastructure.
- Facilitates load balancing and enhances system resilience by allowing clients to discover all healthy instances of a service and distribute requests among them.
- Eliminates the need for manual reconfiguration or calling external systems when service locations change, such as an IPAM.

Note

While Nomad Enterprise provides basic service discovery capabilities, these cover services running within the Nomad cluster itself. Nomad does not offer native support for external service discovery, ingress routing, or a DNS interface.Enable comprehensive service discovery and routing, including for external clients, by integrating Nomad Enterprise with additional tooling that specialize in service discovery such as Consul, which provides robust service discovery, a DNS interface, and a mature integration pattern with Nomad Enterprise.

8.1 Health checks

Health checking is a crucial component of service discovery and management in Nomad Enterprise. It ensures that only healthy, properly functioning service instances are available for discovery and available other applications. Service health checks represent tests that Nomad Enterprise performs to determine the status of a service.

Visit the Service Discovery on Nomad tutorial for a guide how to implement it into your job file. Use the `check` and `service` documentation pages as reference for a detailed list on the various options available.

Nomad supports the following capabilities for native service discovery.

> **Note**
>
> Specify `provider = "nomad"` as it defaults to Consul when using the `service` block.

8.1.1 HTTP health checks

This example configures an HTTP health check for a web application. It sends a GET request to the `/healthz` endpoint every 10 seconds and if receiving no response, is timed out after 5 seconds.

```
service {
  name = "web-service"
  port = "http"
  provider = "nomad"

  check {
    type     = "http"
    path     = "/healthz"
    interval = "10s"
    timeout  = "5s"
    method = "GET"
  }
}
```

If your application has multiple HTTP endpoints, you can use multiple HTTP service checks. Also, you can add in custom headers. This example shows two services.

1. HTTP GET request to the `/healthz` endpoint over port 8080 using a `Authorization` header.

2. HTTP GET request to a `/v1/db-check` to ensure the API service is able to communicate with the database.
3. If both status return a `200` code, then the health check passes. If one fails, they both fail.

```
group {

    network {
        port "api" {
            to = 8080
        }
    }

task "api" {

#...

  service {
    name = "api-service"
    port = "api"
    provider = "nomad"

    check {
      type     = "http"
      path     = "/healthz"
      interval = "10s"
      timeout  = "5s"
      method = "GET"
      header {
        Authorization = ["Basic ZWxhc3RpYzpjaGFuZ2VtZQ=="]
      }
    }
  }
  service {
    name = "db-ready"
    port = "http"
    provider = "nomad"

    check {
      type     = "http"
      path     = "/v1/db-check"
      interval = "10s"
      timeout  = "5s"
      method = "GET"
    }
  }
```

Use multiple checks to verify different components or functionalities of your service and adjust intervals based on the importance and expected response time of each check.

> **Note**
>
> Failure is inferred from any non-2xx response code. Tweak your application appropriately to respect this.

8.1.2 TCP health check

You can, instead, use a TCP health check for services where a straightforward connection test is sufficient. Adjust the interval based on the criticality of the service and acceptable detection time for failures. This example attempts to establish a TCP connection to the label *endpoint* port defined in `group.network{}` every 10 seconds. The check fails if the connection is not established within 5 seconds.

```
  service {
    name = "app"
    port = "endpoint"
    provider = "nomad"

    check {
      type     = "tcp"
      port     = "endpoint"
      interval = "10s"
      timeout  = "5s"
    }
  }
```

- If the application uses multiple ports, use multiple service checks. Remember, if one check fails, they both fail.

```
group {

    network {
        port "endpoint1" {
            to = 8080
        }
        port "endpoint2" {
            to = 9000
        }
    }

task {

#...

  service {
    name = "app_endpoint1"
    port = "endpoint1"
    provider = "nomad"

    check {
      type     = "tcp"
      port     = "endpoint"
      interval = "10s"
      timeout  = "5s"
    }
  }
service {
    name = "app_endpoint2"
    port = "endpoint2"
    provider = "nomad"

    check {
      type     = "tcp"
      port     = "endpoint2"
      interval = "10s"
      timeout  = "5s"
    }
  }

#...
 }
}
```

8.2 Upgrades

Use health checks during upgrades to ensure the application is healthy before promoting it. When performing an upgrade, the `health_check = "checks"` directive within the `update` block means Nomad Enterprise respects the service checks of the tasks. If the health check fails, then a roll back occurs.

```
job {
  update {
      health_check = "checks"
      auto_revert = true
  }
  group "frontend" {
    count = 3

    network {
        port "http" {
            to = 8080
        }
    }
    task "web" {
        service {
            name = "web-service"
            port = "http"
            provider = "nomad"

            check {
              type     = "http"
              path     = "/healthz"
              interval = "10s"
              timeout  = "5s"
              method = "GET"
            }
        }
    }
  }
}
```

8.3 Leveraging service discovery in tasks

In order for services to communicate with each other without having to know their IP address or DNS host name you need to leverage the template feature. Templates along with service discovery, offers a way to dynamically configure tasks based on the current state of your infrastructure. These

templates are initially rendered when a task starts and update either periodically or in response to specific events, ensuring your tasks always have the most up-to-date service information.

For service discovery, the data that is dynamically retrieved are the IP address and port of another task. Here is an example diagram to visualize the templating flow:

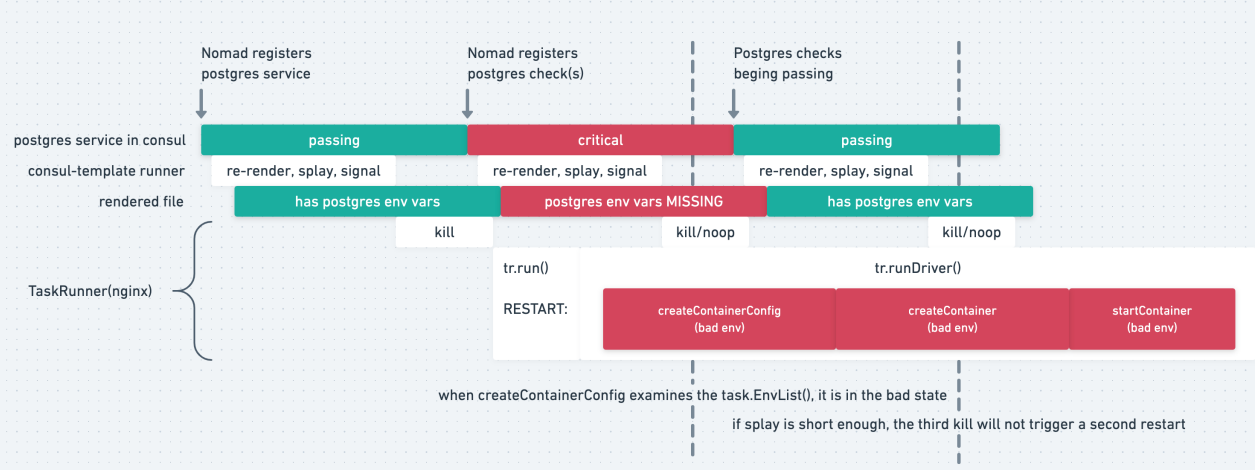


Figure 6: Template rendering flow

Below are two different example tasks that need to communicate with each other.

App1 has a dependency on App2 and is using a hard-coded IP as an environment variable. This does not scale, breaks when Nomad Enterprise schedules App2 on a different host and pins traffic to one instance if there are multiple instances. This poses additional problems if you are using dynamic ports with App2 which can also change whenever restarted or rescheduled.

```
job "app1" {
  group "app1group" {
    network {
        port "app1" {}
    }
    task "server" {
      env {
          APP2_IP = "172.16.30.5"
      }
      service {
            name = "app1"
            port = "app1"
            provider = "nomad"
      }
    }
  }
}
```

```
job "app2" {
  group "app2group" {
    count = 3
    network {
        port "app2" {}
    }
    task "server" {
      service {
            name = "app2"
            port = "app2"
            provider = "nomad"
      }
    }
  }
}
```

To fix this, we can put the environment variable as a template to ensure we always have the correct IP and port of App2.

```
job "app1" {
  group "app1group" {
    network {
        port "app1" {}
    }
    task "server" {
      template {
          data = >>EOH
{{ range nomadService "app2" }}
        APP2_IP = {{ .Address }}:{{ .Port }}
{{ end }}
        EOH
        destination = "local/vars.env"
        env = true
      }
      service {
            name = "app1"
            port = "app1"
            provider = "nomad"
      }
    }
  }
}
```

By default, if there is a change to the App2 IP address port, Nomad Enterprise by default automatically restarts the App1 task. Use the `change_mode` to determine what action occurs when Nomad Enterprise detects a change with the rendered template.

Templating is a powerful tool which allows a lot of creativity and flexibility for your workloads. Visit `template` documentation page for more information.

9 Secrets and variables

9.1 Understanding and implementing Nomad Enterprise variables and secrets

Nomad Enterprise provides a way to store and manage variables. It uses a path-based structure for organizing and accessing data, similar to HashiCorp Vault and replicates between servers.

Use different paths within Nomad Enterprise to store variables, including job, group, and task levels, or any custom path. Variables defined at the job level are not accessible to other jobs. Nomad Enterprise isolates group-level variables to their specific group, and task-level variables are only

accessible by specific tasks. Along with ACLs, this scoped accessibility helps maintain the principle of least privilege and reduces the blast radius for compromised variables.

Nomad variables are useful for storing sensitive information such as API keys, credentials, and other small pieces of data that tasks may require in a key-value format. They are truly encrypted, not merely base64 encoded. Encryption ensures that sensitive information remains secure both at rest and in transit.

The encryption keys are managed by Nomad, providing an additional layer of security compared to storing secrets in plain text or using weak encoding methods.

9.2 Nomad versus HCL variables

Nomad Enterprise supports two types of variables: Nomad variables and HCL (HashiCorp Configuration Language) variables. While both serve the purpose of storing and using data within Nomad Enterprise, they have distinct characteristics and use cases.

Nomad variables are static k/v formatted variables. Nomad Enterprise stores, encrypts and replicates these and they be dynamically accessed at runtime by your application. Replication makes them suitable for storing sensitive information such as API keys or credentials. Use the command-line tool, API, or web UI to create, update, and delete Nomad variables.

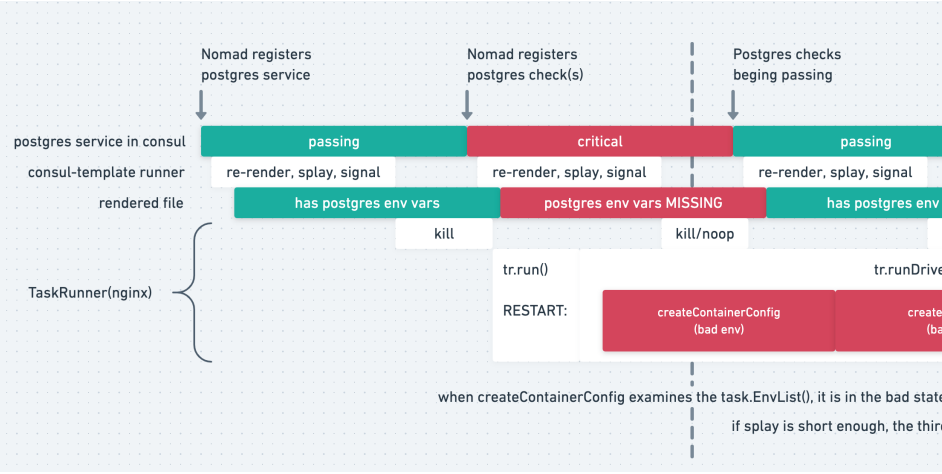
Example use of Nomad variables

Access variables by using the `template` and nomadVarList function.

```
template {
  data        = <<EOH
{{ range "path/to/filter/" }}
  {{ . }}
{{ end }}
EOH
}

template {
  data        = <<EOH
{{ range nomadVarList "nomad/jobs/somejob" }}
  {{ .Path }}
{{ end }}
EOH
}
```

Because of the use of templates, the job does not need to be resubmitted to update the variable value. Nomad Enterprise will use the `template` to detect a change in the variable and automatically restart by default, which makes it ideal for application level configuration data. Here is a diagram to show the flow of the templating engine:



9.3 HCL variables

Job specifications define HCL variables and typically use them for configuration values known at the time of job submission. These variables are part of the job file and Nomad Enterprise processes these when parsing the job, so any changes to the variable require resubmitting the job which redeploys everything within the job. This makes it more suitable for job file data over application data such as defining a job name or a namespace.

For more information, visit Input Variables documentation page.

Example use of HCL variables

Define HCL variables just like Terraform configuration.

```
variable "image" {
  description = "The docker image to use"
  type        = string
  default     = "someimage:1.5.3"
}

job "myapp" {

#...

task "app" {
    driver = "docker"
    config {
        image = var.image
    }

#...
```

9.4 Choosing between Nomad variables and HCL variables

When deciding between Nomad variables and HCL variables, consider your existing workflows and the specific needs of your deployment. If you are already familiar with Terraform workflows, using HCL variables in Nomad Enterprise can ease the adoption process, as you do not need to learn a new system for managing variables.

The Nomad job specification allows job files to be readable and understandable. Use HCL variables for non-sensitive configurations like image tags or defining a namespace is often sufficient. However, for sensitive data or values that need to be dynamically updated, Nomad variables offer better security and flexibility.

If you require dynamic templates for your CI/CD pipelines, consider using Nomad Pack instead of extensively using HCL variables. Over-reliance on HCL variables can limit your pipeline's scalability and flexibility in the long run.

Here is a table summarizing the differences:

	Nomad variables	HCL variables
Storage	Nomad's state store	Job specification and Git repository

	Nomad variables	HCL variables
Security	Encrypted in Nomad's state	No Encryption & must use another tool to hide secrets
Access	Dynamic, at runtime	At job parsing
Management	Command-line tool, API, Web UI	Job file editing
Scope	Share across jobs	Job-specific
Use case	Sensitive data, dynamic values	Job file configuration values
RBAC	Fine-grained ACL policies	Use Sentinel to enforce
Updates	Update without job modifications	Require job file updates and resubmit job

9.5 Nomad variables versus Vault variables

Using Nomad variables for secrets is beneficial for organizations in the early stages of adoption or those seeking a straightforward secrets management for your applications without needing to implement additional systems until later.

Storing secrets within Nomad Enterprise is ideal for small static secrets such as API keys, credentials, or configuration values that do not change frequently. Nomad variables consume server resources and are not designed to handle large volumes of changing data such as batch job results. As such, scaled organizations or those with dynamic secret needs must consider using Vault for more robust and scalable secrets management.

Offloading secrets management to Vault not only provides enhanced capabilities but also frees up compute resources on the Nomad server nodes. However, it is important to note that a dedicated secrets management solutions like HashiCorp Vault exceeds the capabilities of Nomad's. Nomad Enterprise does not support dynamic secrets generation or advanced features such as PKI (Public Key Infrastructure) management.

While Nomad Enterprise's built-in secrets management is sufficient for many use cases and provides an excellent starting point. Consider implementing Vault alongside Nomad if you have more complex requirements or have a growing Nomad Enterprise footprint.

In short, storing Nomad variables as secrets is a great option for static secrets and ease of adoption, but for dynamic secrets, PKI, and more advanced secrets management features, Vault is the recommended solution.

9.6 How to use variables

9.6.1 Creating a variable

You can create and store a variable in three different ways: command-line tool, API, and UI. You can define variables at different levels - job, group, or task. This hierarchical approach lets you control access to variables, limiting their visibility to specific scopes. For instance, a variable created at the task level is only accessible within that task, while a job-level variable is usable across groups and tasks within that job.

Store variables you may want accessible by all jobs in a path of your choosing. If you do this, ensure the variable either is appropriate to be globally accessed or if not, it has the appropriate ACL policies attached.

Command-line tool

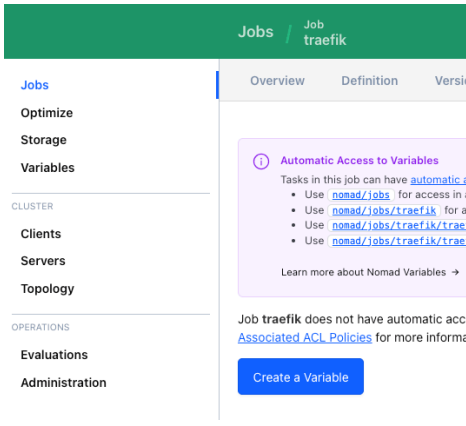
Store variables tutorial covers how to create and store variables

API

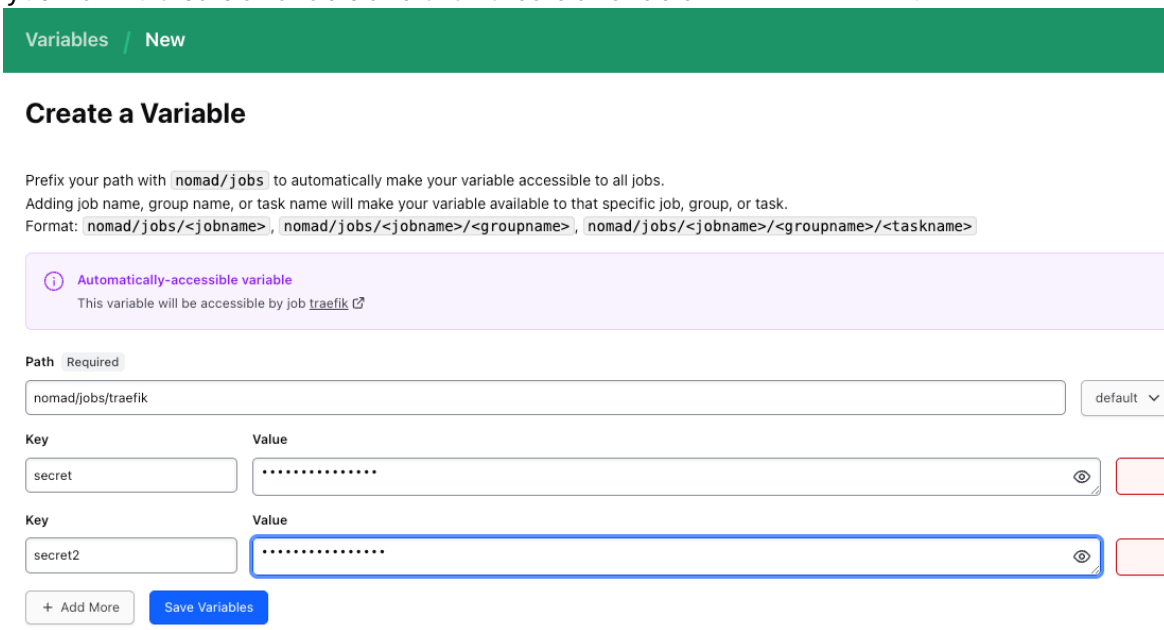
`v1/var/` endpoint creates, reads, and lists variables

UI

Go to the job for which you want to create a variable and click *Create a variable*



Input the required k/v
Click “Save Variables”



9.6.2 ACLs

Always protect access to variables with Access Control Lists (ACLs).

The workload identity for each task automatically grants read and list access to variables found at Nomad-owned paths with the prefix `nomad/jobs/`, followed by the job ID, task group name, and task name.

We recommend creating proper ACL policies for all variables stored outside the standard workload

identity created ACLs. Visit the Identity and Access Management portion of the Administration Guide, Task Access to Variables for more information on adding additional policies for a particular variable path or the Access Control tutorial.

9.6.3 Accessing variables from tasks

Access variables and secrets from tasks by using `template` and `nomadVarList` function, however if you want a step by step tutorial visit the Access Variables tutorial.

We recommend rendering templates containing sensitive information in the `secrets/` task working directory, which offers enhanced protection by being inaccessible outside the task, and unreadable to the API, command-line tool, or stored in-memory. For variables that are not sensitive, optionally store these under `local` or `tmp` directory. Nomad Enterprise uses plain text for these.

For more information on the filesystem architecture of Nomad, visit the Filesystem concepts page.

Secret environment variable example

This example shows accessing a variable from the `api-app` job path and assigning the value to the `API_KEY` environment variable. Nomad Enterprise renders this to the tasks `secrets/` directory.

Groups and tasks within the job can access the `api-key` variable and no other jobs or requests outside of the job can access it. The rendered template is only accessible to the task itself.

```
template {
  data        = <<EOH
{{ range nomadVarList "nomad/jobs/api-app" }}
  API_KEY = {{ .api-key }}
{{ end }}
EOH
    destination = "secrets/vars.env"
    env = true
}
```

Non-sensitive environment variable example

This example shows a variable we store under the `nomad/jobs` path, which makes it accessible to all jobs within Nomad.

```
template {
  data        = <<EOH
{{ range nomadVarList "nomad/jobs" }}
  VAR = {{ .key }}
{{ end }}
EOH
    destination = "local/vars.env"
    env = true
}
```

Configuration file example

Nomad secrets and variables are not only for environment variables. Also use these for configuration files.

```
template {
    data        = <<EOH
<!DOCTYPE html>
<html lang="en">
  <head><meta charset="utf-8"><title>Hello Variables</title></head>
  <body>
    <p>The task has access to the key variable! The key variable is:</p>
    <ul>

      {{- range nomadVarList "nomad/jobs/nginx" }}
      <li>{{ .key }}</li>
      {{- end }}

    </ul>
    <p>[View the index page.]()</p>
  </body>
</html>
EOH
        destination = "local/index.html"
      }
```

9.6.4 Locks

Nomad Enterprise can block the update of a variable for a period of time by setting a lock on it. Upon locking a variable, everyone can read it, but only the lock-holder can update it.

Variable locks are great for stateful applications clustered together that utilize leader election. View the variable locks concepts document for more details.

10 Changelog

10.1 2026-08

- Restructured legacy Nomad HVD content into the HVD 2.0 Nomad User Guide as part of the Installation / Administration / User guide migration.

10.2 2026-01

- Update meta descriptions
- Updated credits and contributors
- Fix threshold descriptions

10.3 2025-10

- Review of Nomad Operating Guide

10.4 2025-07

- Updated documentation links

10.5 2025-05

- Removed document structure section
- Fixed code block formatting

10.6 2025-04

- Updated documentation links

10.7 2025-02

- Update introduction

11 Credits

Thank you to everyone who contributed to this validated design guidance. This cross-functional team, representing many departments within HashiCorp, authored, edited, reviewed, and iterated this content to provide prescription and recommendations for using HashiCorp software in enterprise scenarios.

- Adrian Todorov
- Amber Reddy
- Andrei Burd
- Mark Lewis
- Prakash Manglanathan
- Shaun Coss